

Memoria Práctica 3 — Q-learning para seguimiento de línea

1. Introducción

Controlador en Python para el Khepera IV en Webots. El robot aprende a seguir una línea negra mediante **Q-learning tabular** sin modelo del entorno, complementado con un módulo reactivo de evitación de obstáculos.

Código: `controllers/p3_controller/p3_controller.py`.

2. Estados, acciones y recompensa

Estados (3). A partir de los 4 IR de suelo y los umbrales > 750 (negro) y < 500 (blanco):

- **S1:** línea perdida por la izquierda (central izq. negro, lateral der. blanco).
- **S2:** línea perdida por la derecha (central der. negro, lateral izq. blanco).
- **S3:** resto de combinaciones.

Esta discretización deja una Q-table 3×3 que converge rápidamente.

Acciones (3). Girar derecha (A1), girar izquierda (A2) y avanzar recto (A3). Los giros se implementan como arcos dinámicos (rueda interna a 1 rad/s, externa a 15 rad/s) en lugar de pivotes en el sitio, evitando bucles parásitos sin desplazamiento real. La duración es asimétrica: 1 tick para avanzar y 2 ticks para girar, de modo que las correcciones de rumbo cuestan más tiempo de simulación que el avance, haciendo rentable ir recto en S3.

Recompensa. Derivada de la experiencia, comparando para cada sensor su lectura antes y después de la acción. Los pesos `[1.0, 2.0, 2.0, 1.0]` priorizan los sensores centrales. Las transiciones blanco→negro, negro→negro, negro→blanco y blanco→blanco aportan $+0.5 \cdot w$, $+1.25 \cdot w$, $-0.75 \cdot w$ y $-w$ respectivamente. Además, un bonus de ± 1.0 se suma cuando los 4 sensores están simultáneamente en negro (centrado) o en blanco (totalmente fuera), desambiguando S3.

3. Aprendizaje

Actualización. Se aplica literalmente la regla no-determinista del enunciado:

$$Q_n(s,a) \leftarrow (1 - \alpha_n) \cdot Q_{\{n-1\}}(s,a) + \alpha_n \cdot [r + \gamma \cdot \max_{a'} Q_{\{n-1\}}(s',a')]$$

con $\alpha = 1/(1 + \text{visits}[s,a])$, $\gamma = 0.5$ y la Q-table inicializada a ceros en cada arranque (sin persistencia).

Exploración vs. explotación. Política ϵ -greedy: con probabilidad ϵ se elige una acción aleatoria; en caso contrario, `argmax_a Q[s,a]`. El parámetro ϵ decae linealmente de 1 a 0 a lo largo de 500 iteraciones, de modo que el agente pasa de explorar puramente a explotar la política aprendida.

4. Evitación de obstáculos

Módulo reactivo no aprendido sobre los 3 IR frontales (FL, F, FR). El central dispara a 250 (detección a distancia) y los laterales a 500 (sólo cuasi-contacto), evitando falsos positivos con paredes paralelas. Cuando se activa, el robot gira hacia el lado más despejado: 45° si el disparo es central y 10° si es lateral, midiendo la rotación con los encoders de las ruedas. Tras la maniobra se releen los sensores de suelo y el bucle Q-learning retoma el control.

5. Bucle principal

1. Inicializar motores, encoders, sensores y Q-table.
 2. En cada iteración: `avoid_obstacles()` ; calcular ϵ y elegir acción; leer suelo previo, ejecutar la acción y leer suelo posterior; calcular recompensa y estado siguiente; actualizar la Q-table.
-

6. Conclusiones

El controlador cumple los requisitos: Q-learning tabular con recompensa derivada de la experiencia, tasa adaptativa y transición progresiva de exploración a explotación. Las decisiones clave fueron sustituir los giros estáticos por arcos dinámicos para eliminar bucles parasitarios, establecer duraciones asimétricas que favorecen el avance recto en S3, añadir el bonus de configuración completa para desambiguar S3, y calibrar umbrales y ángulos diferenciados en la evitación para reducir falsos positivos y ajustar la respuesta al grado de amenaza.

Durante el desarrollo se observó que, aunque velocidades bajas (~4 rad/s) eran más estables, el comportamiento a 15 rad/s —tras ajuste de parámetros de movimiento y recompensa— resultó aceptable, si bien requiere ocasionalmente reiniciar la simulación hasta obtener una ejecución con aprendizaje convergente.